SERVER OPERATIONS SPECIALIST, JC-520957

Interview Answer Sheet, Panel Thursday July 23 2026, 2:00 p.m.

The seven most likely questions, read for shape and flow, not for word-for-word memorization

Question 1 of 7 · Why you're qualified

Tell us why you think your training, education, skills, and experience make you the most qualified person for this job.

The clearest proof is a highly available AWS environment I designed and built myself, two availability zones, a self-healing network path, and a Multi-AZ database for backup and recovery. I tested it by actually breaking it, 45 failure simulations across three full runs, about two and a half minutes average recovery, one run logging 92 events with 15 automatic route repairs and zero false positives. That's the habit this role needs, not just standing a system up, but proving it holds when something fails.

I've got the same track record closing gaps on a live system. In my internship I took a Windows Server from about 30 vulnerability findings down to 3, tracing each one to its root cause and clearing every critical by the second scan. I also ran a DISA STIG compliance audit that came back with 142 failing checks out of 263, picked the 10 that actually mattered for that environment, remediated them with PowerShell, and verified every one with a re-scan.

Underneath both of those is my B.S. in Network Engineering and Security from WGU, plus CompTIA A+, Network+, Security+, and Cloud+. The credentials gave me the theory, the projects are where I actually tested it against something real.

Everything I build comes back to confidentiality, integrity, and availability, keeping a system trustworthy and available, not just running. I haven't had a job with 'server administrator' in the title yet, so there's a real ramp ahead on your specific tools and scale, but the habit of verifying my own work instead of calling it done is already built in.

Question 2 of 7 · Responsibilities overview

Please describe in detail what you believe are server system specialist responsibilities.

I think the responsibilities start with one question, what is this server actually for. A database server, a web server, and an internal file server don't carry the same risks, so the job shifts with the workload. What stays constant is the goal underneath all of it, and I think of that through the CIA triad, keep the data confidential, keep it intact, and keep it available.

Confidentiality is making sure the server and its data are only reachable by the people and systems that should reach them, authentication, access control, locking down how you connect to the box in the first place. Integrity is the data staying accurate and uncorrupted, which is where audit logging and change control matter. Availability is the one that takes the most engineering, redundancy, backups you've actually tested, and a real plan for when something fails.

Around that core sit the daily duties, designing the server and its environment, installing and configuring it, keeping it running through maintenance and updates, monitoring it, troubleshooting to the actual root cause when something breaks, patching it, tracking when hardware and software hit end of life so you can plan the replacement before it's a problem, and documenting the work so the next person, including future me, can understand the system.

I built a two-availability-zone AWS environment to put the availability piece into practice specifically. I used self-managed NAT instances instead of a managed NAT Gateway to save cost, then engineered the reliability back in myself, a Lambda that automatically repairs the network route when a NAT instance gets replaced. It runs a Multi-AZ database, access is SSM-only with no SSH keys, and I tested the whole thing by simulating failures and measuring recovery. Building that project is really where I learned that designing for availability mostly means planning for the failure you hope never comes.

Question 3 of 7 · Patch management

Describe your experience with patch management and system updates.

I work with patch management in two places, a hands-on internship lab and my own homelab.

In the internship I run vulnerability and compliance scans against Windows and Linux machines in Microsoft Azure using Tenable, pull the findings, and remediate what I can with PowerShell. Across several rounds I took one Windows Server from about 30 findings down to 3, clearing every critical along the way.

The three that were left weren't loose ends. Two were self-signed SSL certificate findings, expected in a lab using internally issued certificates. The third was a SQLite finding that kept flapping, showing on one scan, dropping off the next, then coming back. Instead of remediating it on reflex, I searched the system with PowerShell for the actual SQLite files and found none installed, which told me it was a false positive, so I documented it as one instead of forcing a change the machine didn't need. I learned verification cuts both ways from a separate case, an Edge update I remediated that re-flagged on the next scan. Edge had actually downloaded the update, it just needed to be closed and reopened before it applied. Once I did that and re-scanned, it came back clean. So now I confirm a real fix landed the same way I confirm a noisy finding is real before I act on it.

At home that habit is just daily practice instead of a project. My Linux hosts run unattended upgrades automatically, but I usually patch by hand too, I wrote a short script that runs update, upgrade, autoremove, and autoclean, aliased to 'up' so I can run it from anywhere, and I run that across my servers, two Pi-holes, and my pfSense firewall. The habit underneath all of it is the same one, find out what's out of date or misconfigured, fix it the same way every time, and re-scan to actually prove it worked.

Question 4 of 7 · Troubleshooting / root cause

Describe how you troubleshoot a server issue and conduct root cause analysis, including when you'd escalate it.

Root cause, I'll start with an example from a lab environment at WGU. Half a department kept intermittently dropping off the network, some machines on, some off, changing through the day. I traced it to an exhausted DHCP scope, no free addresses left to lease. The quick fix would have been widening the scope and moving on, but that just delays the same failure showing up again. I dug into why the scope was exhausted in the first place and found leases were held for a full month, so addresses from machines that were long gone were still tied up. I shortened the lease time so stale addresses recycle in days instead of a month. That's the habit, fix the cause, not just the symptom sitting in front of you.

Knowing when something's bigger than it looks is the other half of this. Another WGU lab ticket, a user reported one site loading the wrong page. Before I touched anything, I asked a few other people to try it too, and it failed for all of them, which told me it wasn't that one machine, it was shared infrastructure. That pointed me at DNS instead of the endpoint, and I found a rogue forward lookup zone redirecting the domain and removed it. The reproduction step is what told me this wasn't a one-user problem.

Where I'll be honest is I've never had a job with a formal escalation chain to hand something up to, my internship is remote and self-directed, so there's no supervisor I've actually escalated a live issue to. What I do understand is the structure of it. I built a self-hosted ticketing system with an SLA and an escalation tier that automatically flags a ticket and bumps its priority a day before it's due, so I know how escalation is supposed to work mechanically, even without having lived it on the job. My own rule for when I'd escalate is simple, when it's outside what I have the access or authority to fix, or when it's clearly bigger than one system and needs more than standard procedure covers.

Question 5 of 7 · Backup and recovery

Explain your experience with backup and recovery, and how you'd verify a backup is actually recoverable.

When I designed the database and network path for my AWS environment, I didn't want recoverability to be something I just hoped worked. I put the database on Multi-AZ, so there's a synchronous standby ready to take over automatically if the primary fails, and I built a repair Lambda that watches the network path and fixes it the same way.

But a design on paper isn't proof. So I tested it by actually breaking it, terminating instances across every failure combination I could think of, 45 times across three full test runs. That's where I learned something specific, the repair Lambda detects a failure and fires in well under a second, but the full recovery, AWS replacing the terminated instance and the network settling back to normal, averaged about two and a half minutes across all 45 runs. Knowing exactly where that time goes, not just that it eventually works, is what I'd call actually verifying a recovery instead of assuming one.

What I'll be upfront about is I haven't run an enterprise backup product day to day. I don't have hands-on time with whatever you use here, and I'd want to learn your retention policy and restore procedure early. But the instinct is the same one I'd bring to that, a recovery plan is only real once you've triggered the failure yourself and timed what actually happens, not before.

Question 6 of 7 · Difficult conversation

Tell us about a time you had to communicate something delicate to a coworker or stakeholder, or handle a difficult one.

As a tax preparer, once a client discloses something, I'm legally obligated to act on it, I can't just let it go. An older client mentioned she lived alone in her own home. When I asked whether she rented or owned, she deflected, but I couldn't drop it, so I explained that if she owned the home she'd have gotten a 1098 mortgage interest statement we needed to file accurately. She admitted she owned it and said she'd look for it, but never produced it.

She decided she didn't like me over it. I stayed calm and factual, told her it was completely fine to work with another preparer, and handed her off with no hard feelings. The next preparer asked for the same document for the same reason, she accepted it from them, and the return got filed correctly. The part I care about is the outcome being right, not being liked in the moment.

Question 7 of 7 · CUIAB / motivation

What do you know about CUIAB, and why do you want to work here?

CUIAB is the appeals board for EDD's unemployment, disability, and paid family leave decisions, so it's a due-process function, not a benefits office, an administrative law judge hears the first appeal, and the Board reviews it if it goes further. I read your 2023-2027 strategic plan, and what stood out was the focus on modernizing legacy infrastructure and moving toward faster, more accessible service, especially after the claim volume the pandemic drove through appeals boards statewide. That's exactly the kind of work the lifecycle-management piece of this role is about, knowing what's aging out before it becomes the thing that threatens reliability for someone waiting on a hearing. That's a mission I want to be behind, not just a paycheck.